Lab 8

Number Bases

CSC 2001 - Data Structures

Overview

A number does not have a base - only the way we *write* it does. 45, `101101` in base 2, `55` in base 8, and `2D` in base 16 are all the same number. In this lab you will write the conversions from lecture yourself, in a class called `BaseConverter`: reading a string of digits in any base from 2 to 36 into an `int`, and writing an `int` back out as digits in any base.

You will write the conversion to digits twice: once with a loop and a **stack**, and once with **recursion** and no stack - and then explain why the two are really doing the same thing.

Learning goals:

- Read digits left to right with value = value × base + digit, and detect overflow past `Integer.MAX_VALUE`.
- Produce digits with repeated division (`n % base` and `n / base`), using a stack to put them in the right order.
- Write the same conversion recursively, and count its calls to see that it is O(log n).
- Validate input and throw `IllegalArgumentException` for bad bases, bad digits, and out-of-range values.
- Test against Java's own conversions with a loop over many values and every base.

Getting Started

Create your repository from the Lab 8 template (remember to make your repo Private):

https://github.com/cp-csc2001/csc2001-lab8-template-s01

The template has the same layout as previous labs. Everything lives in the default (unnamed) package - do not add a package declaration to any file.

File	Contents
`Module/src/main/java/BaseConverter.java`	The BaseConverter class with every method stubbed out (`// TODO`). Constants and private helper signatures are provided, plus the Part 3 questions at the bottom.
`Module/src/test/java/BaseConverterTest.java`	Starter tests plus a list of the minimum tests you still need to write.

One rule: inside `BaseConverter.java`, do not use `Integer.parseInt`, `Integer.toString(n, base)`, `Integer.valueOf`, `Character.digit`, or `Character.forDigit` - the point is to write the conversions yourself. You may (and should) use them in your tests to check your answers.

Background: Digits and Place Value

In base *b*, each position is worth *b* times the position to its right, and the digits run from 0 to *b* - 1. Bases above 10 borrow letters for the digits past 9. This lab supports every base from 2 to 36 - the 10 digits plus the 26 letters:

Characters	Digit values	Characters	Digit values
`'0' - '9'`	0 - 9	`'A' - 'Z'`	10 - 35
		`'a' - 'z'`	10 - 35 (reading only)

Methods that **read** digits accept upper- or lowercase letters. Methods that **write** digits use uppercase. The constant `DIGITS` holds the 36 digit characters in order, so `DIGITS.charAt(13)` is `'D'`.

Reading digits (Part 1) works left to right. Each new digit shifts the value so far one place left (multiply by the base), then fills the ones place:

Digit read	value × 2 + digit	value
1	0 × 2 + 1	1
0	1 × 2 + 0	2
1	2 × 2 + 1	5
1	5 × 2 + 1	11
0	11 × 2 + 0	22
1	22 × 2 + 1	45

Writing digits (Part 2) works by repeated division: `n % base` is the last digit, and `n / base` drops it. The digits come out last digit first, so in lecture we read the remainders from the bottom up. In code, a stack does the reversing: push each digit as it comes out, then pop them all.

n	n / 16	n % 16	Stack after pushing (top on the left)
45	2	13 → 'D'	D
2	0	2 → '2'	2 D
0			stop: pop 2, then D → "2D"

Part 1: Reading Digits - digitValue and toDecimal

`digitValue(char ch)` returns the value of one digit character: `'0'-'9'` give 0-9, and `'A'-'Z'` or `'a'-'z'` give 10-35. For any other character it returns -1. It does not know about bases: `digitValue('7')` is 7 even though 7 is not a binary digit. Use character arithmetic (`ch - '0'`, `ch - 'A' + 10`), not a long chain of `if`s.

`toDecimal(String digits, int base)` returns the value of `digits` read in `base`:

1. Check the base with your private `checkBase(base)` helper, and check that `digits` is not null or empty.
2. Start with `value = 0`. For each character from left to right, get its value with `digitValue`. If it is -1, or is not less than the base, the character is not a digit in this base.
3. Set `value = value * base + digit`.
4. Keep `value` in a `long`. After each step, if it is greater than `Integer.MAX_VALUE`, the number does not fit in an `int`. (An `int` would silently wrap around to a negative number - the overflow gotcha from lecture.)

Leading zeros are allowed: `toDecimal("00101101", 2)` is 45. Signs, spaces, and prefixes are not digits: `"-45"`, `"4 5"` and `"0x2D"` are all invalid. Throw `IllegalArgumentException` if the base is not 2 to 36, if `digits` is null or empty, if any character is not a valid digit in that base, or if the value is larger than `Integer.MAX_VALUE`.

Call	Result	Why
`toDecimal("101101", 2)`	`45`	32 + 8 + 4 + 1
`toDecimal("2d", 16)`	`45`	lowercase is fine: 2 × 16 + 13
`toDecimal("ZZ", 36)`	`1295`	35 × 36 + 35
`toDecimal("7FFFFFFF", 16)`	`2147483647`	Integer.MAX_VALUE: still fits
`toDecimal("80000000", 16)`	`throws`	one more than Integer.MAX_VALUE
`toDecimal("129", 8)`	`throws`	9 is not a base-8 digit

Part 2: Writing Digits with a Stack - digitChar and toBase

`digitChar(int value)` is the reverse of `digitValue`: it returns the digit character for a value from 0 to 35, using uppercase letters (`digitChar(13)` is `'D'`). Throw `IllegalArgumentException` for any other value. `DIGITS.charAt(value)` does the work.

`toBase(int n, int base)` returns `n` written in `base`:

1. Check the base, and check that `n` is not negative with your private `checkNonNegative(n)` helper.
2. If `n` is 0, return `"0"`.
3. While `n > 0`: push `digitChar(n % base)` onto a stack (`Deque<Character> stack = new ArrayDeque<>()`, as in the stacks lecture), then set `n = n / base`.
4. Pop every character off the stack into a `StringBuilder`, and return its string.

The result has no leading zeros and uses uppercase letters. For example `toBase(45, 2)` is `"101101"`, `toBase(45, 16)` is `"2D"`, `toBase(45, 3)` is `"1200"`, and `toBase(Integer.MAX_VALUE, 36)` is `"ZIK0ZJ"`. Throw `IllegalArgumentException` if `n` is negative or the base is not 2 to 36.

Part 3: Recursion - toBaseRec and countDigits

Last lecture we saw that `n % base` and `n / base` split a number the way `first` and `rest` split a list: the last digit, and everything before it. That means the same recursive template works on numbers. Write both methods below with **no loops and no stack**.

`toBaseRec(int n, int base)` returns the same string as `toBase`, recursively:

- If `n` is smaller than the base, it is a single digit: return it as a one-character string. (This covers `n == 0`.)
- Otherwise, return the digits of `n / base` (a recursive call), followed by the digit for `n % base`.

`countDigits(int n, int base)` returns how many digits `n` has in `base`, recursively: a number smaller than the base has 1 digit; otherwise it has 1 more digit than `n / base`. So `countDigits(4725, 10)` is 4, `countDigits(45, 2)` is 6, and `countDigits(0, 10)` is 1.

Both methods throw `IllegalArgumentException` if `n` is negative or the base is not 2 to 36.

Part 4: Testing

The starter tests in `BaseConverterTest.java` must keep their names and pass. Aim for 100% line coverage of `BaseConverter`. Add your own tests so that, between them, your tests cover at least:

- `digitValue`: digits, upper- and lowercase letters, and characters that are not digits - including the characters just before and just after `'0'-'9'`, `'A'-'Z'` and `'a'-'z'` (for example `'/'`, `':'`, `'@'`, `'['`)
- `toDecimal`: lowercase letters, `"0"` and leading zeros, base 2 and base 36, and `Integer.MAX_VALUE` written in a few bases
- `toDecimal` throws for: a digit too big for the base, a non-digit character, null, `""`, a base of 1 or 37, and a value one past `Integer.MAX_VALUE`
- `digitChar`: 0, 9, 10 and 35, and that -1 and 36 throw
- `toBase` and `toBaseRec`: 0, single digits, powers of the base (like 256 in bases 2 and 16), `Integer.MAX_VALUE`, uppercase letters, and that negative n and bad bases throw
- `countDigits`: 0, boundaries like 9 / 10 and 99 / 100, 1024 in base 2, and that negative n and bad bases throw
- a loop over many values of n (all of 0-999 plus some large random ones) and every base from 2 to 36, checking that `toDecimal(toBase(n, base), base)` is n, and that `toBase` and `toBaseRec` match `Integer.toString(n, base).toUpperCase()`

Method Reference

Names, parameters and return types are required exactly as written. All methods are `public static`. You may add private helper methods.

Method	Behavior	Cost
`int digitValue(char ch)`	value 0-35 of a digit character (either case); -1 if not a digit	O(1)
`int toDecimal(String digits, int base)`	value of digits in base, read left to right; IllegalArgumentException for a bad base, null or empty digits, an invalid digit, or a value above Integer.MAX_VALUE	O(d)
`char digitChar(int value)`	uppercase digit character for 0-35; IllegalArgumentException otherwise	O(1)

Method	Behavior	Cost
`String toBase(int n, int base)`	n in base, by repeated division and a stack; "0" for 0; IllegalArgumentException if n < 0 or a bad base	O(log n)
`String toBaseRec(int n, int base)`	same result as toBase, recursively, with no loop or stack	O(log n)
`int countDigits(int n, int base)`	number of digits of n in base, recursively; 1 for 0; IllegalArgumentException if n < 0 or a bad base	O(log n)

d is the number of characters in `digits`. A bad base is anything outside 2 to 36 (the constants `MIN_BASE` and `MAX_BASE`).

Private helpers already declared in the starter code: `checkBase(int base)` and `checkNonNegative(int n)`, which throw `IllegalArgumentException` for a bad base and a negative number.

Submission

Commit and push your work, then submit these two files to Gradescope - "Lab 8 - Section 1":

File	Contents
`BaseConverter.java`	your BaseConverter implementation, with a purpose statement on every method
`BaseConverterTest.java`	the starter tests plus your own tests for every method

File names are case-sensitive and must match exactly.

How it is checked: the autograder compiles your files and runs your own tests (results shown right away), reports your line coverage, and runs a set of instructor tests, including the examples above, every exception case, and a comparison against Java's own conversions for thousands of values in every base from 2 to 36. It also checks that `BaseConverter.java` does not call the library conversion methods listed under Getting Started. Instructor test results are visible once grades are published.